

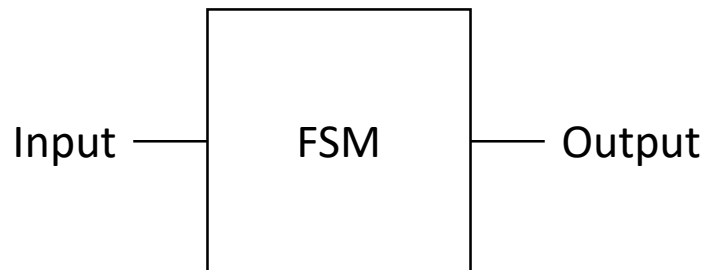
Lecture 11:

Finite State Machines

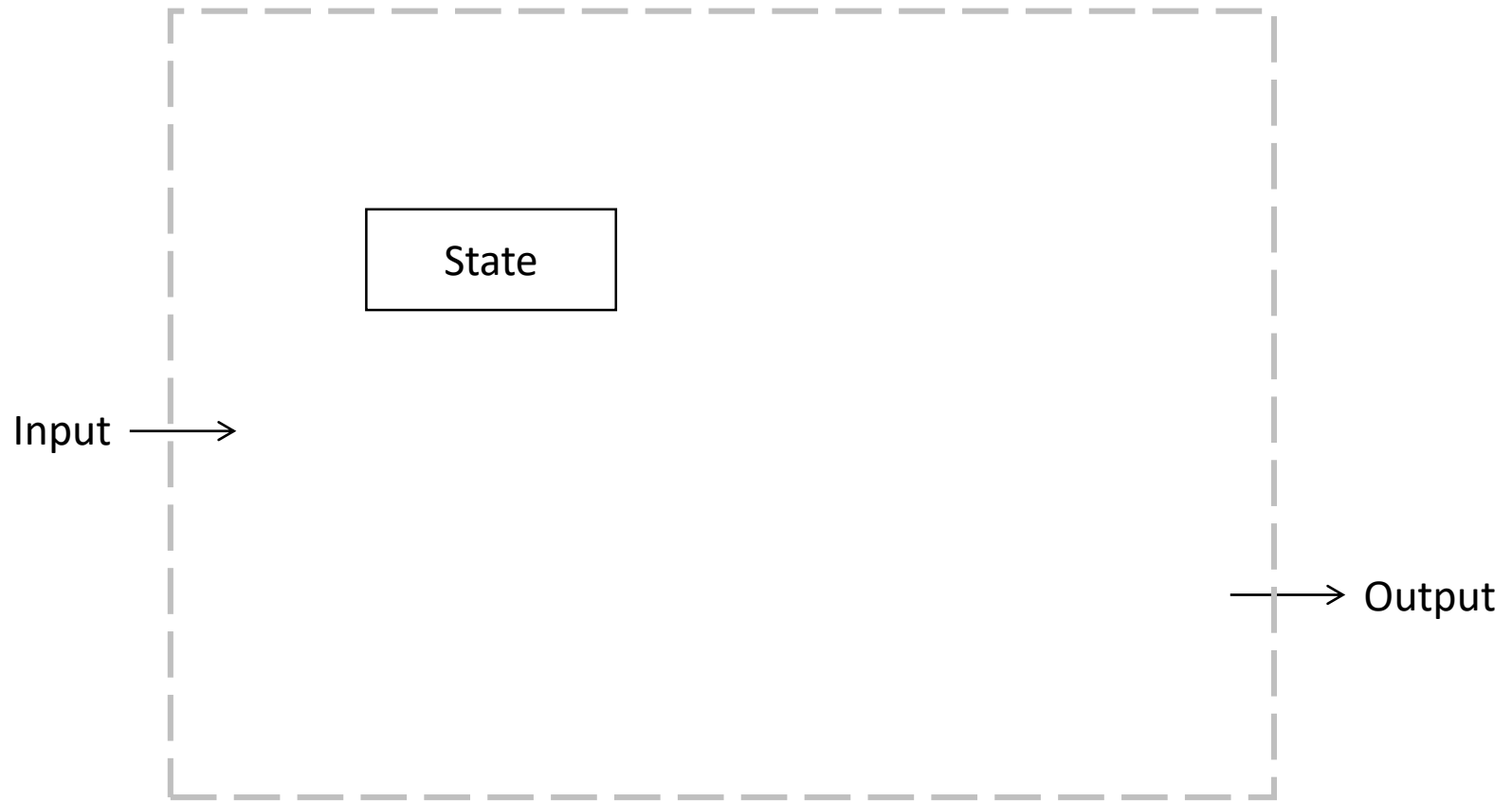
CMPS 221 – Computer Organization and Design

Finite State Machine

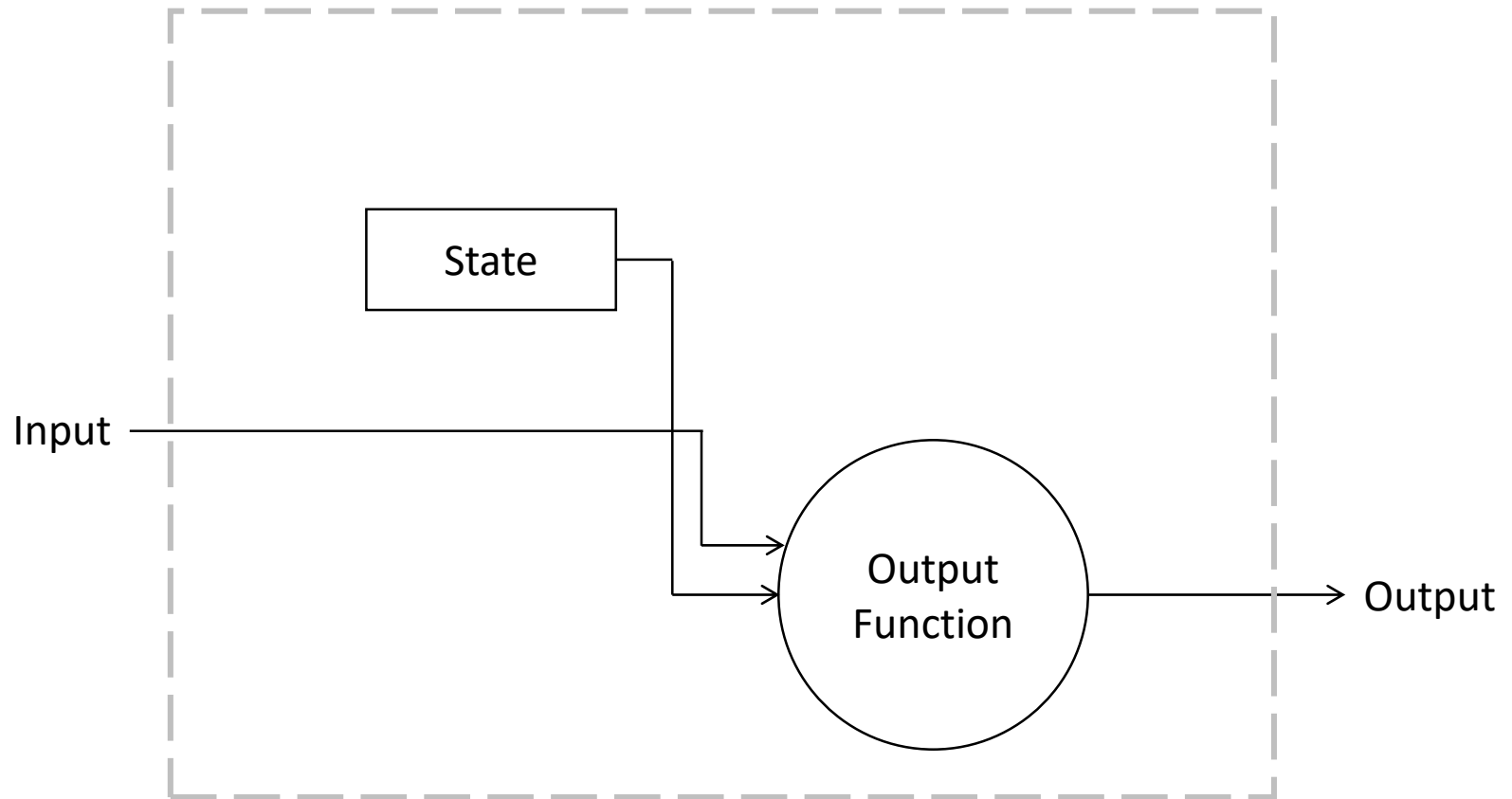
- A **finite state machine** (FSM) is a logic block consisting of inputs, outputs, and internal state
 - It includes an **output function** that computes the output from the inputs and current state
 - It includes a **next-state function** that computes a next state from the inputs and current state



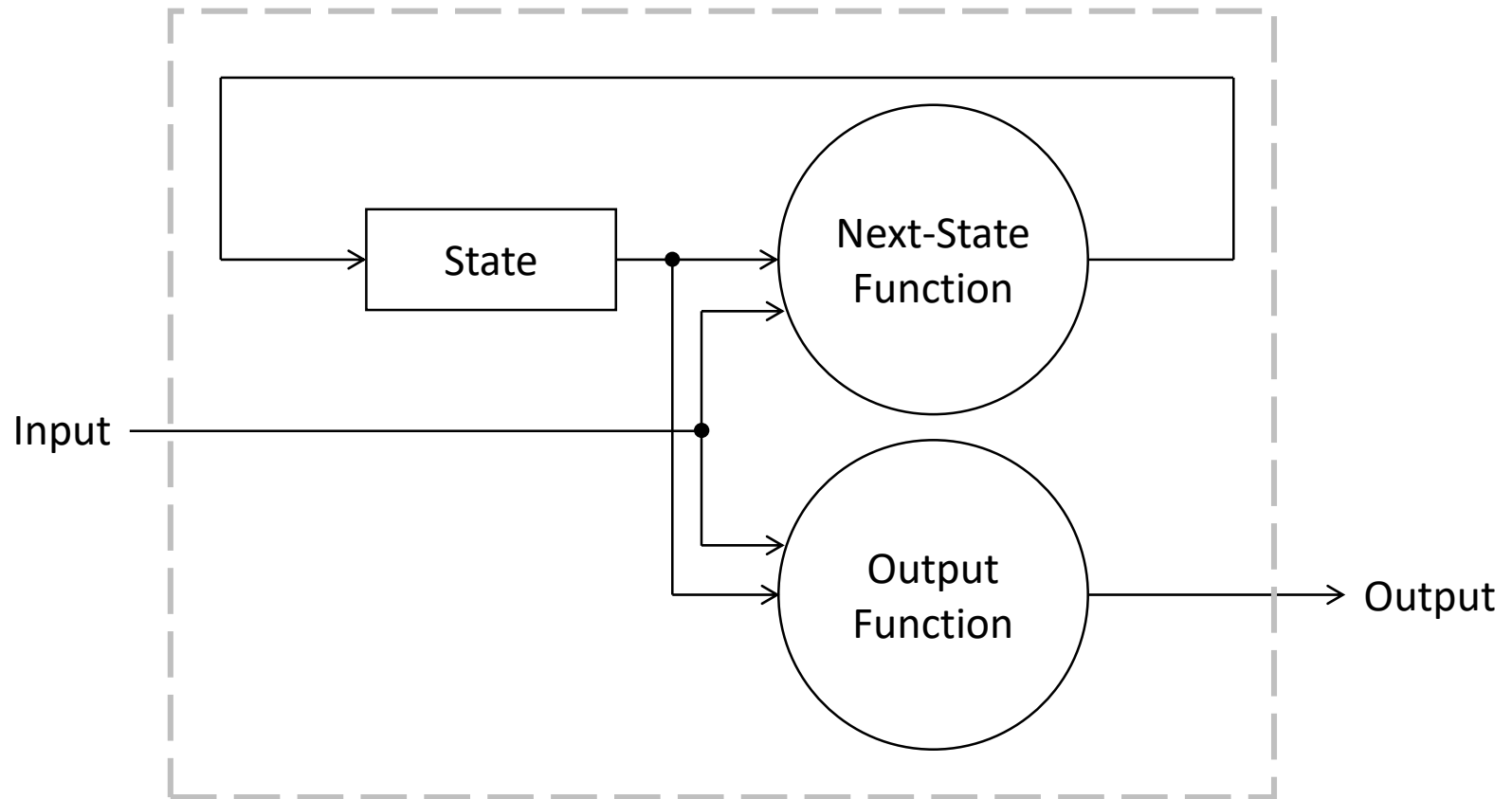
Finite State Machine



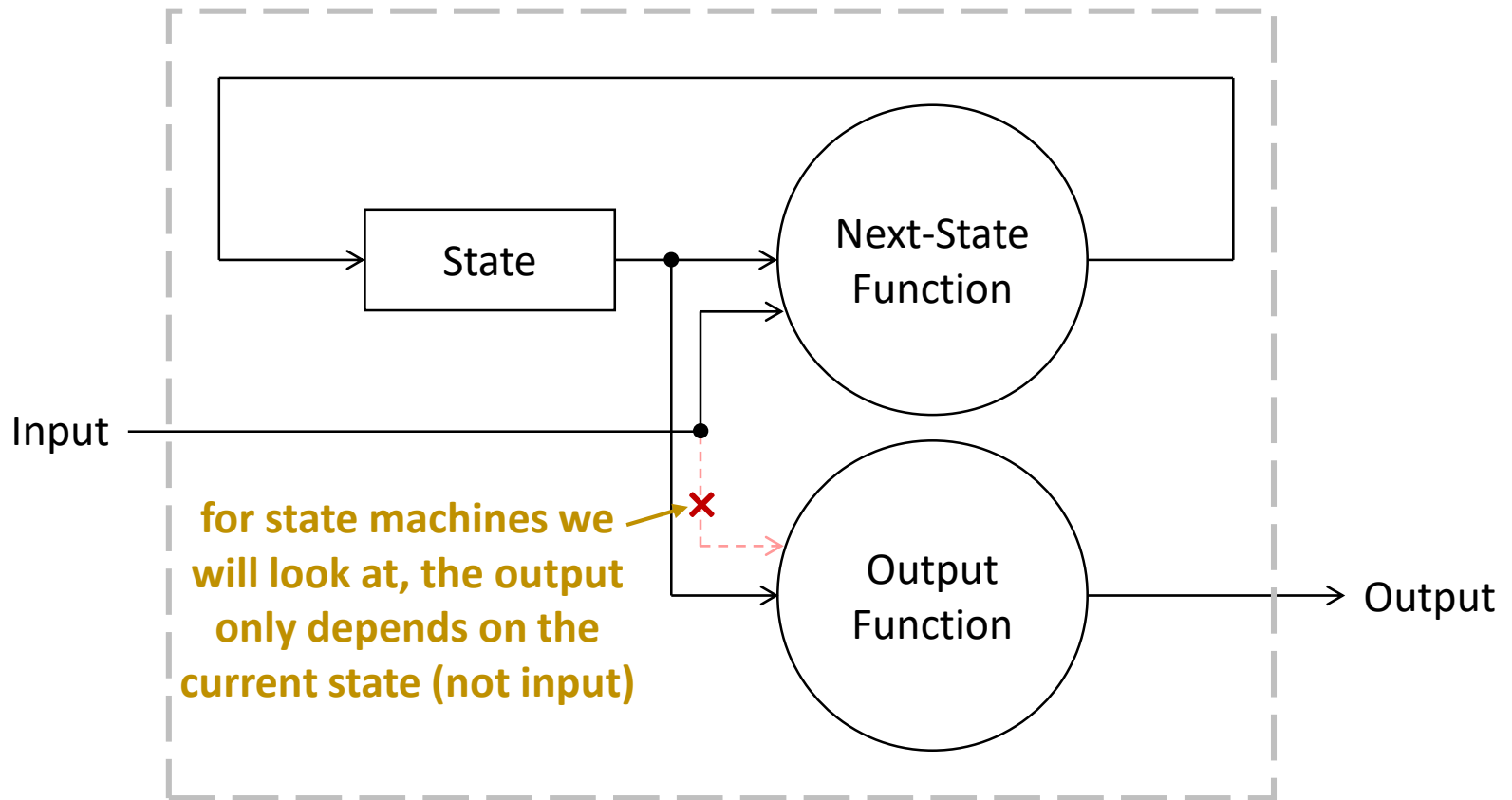
Finite State Machine



Finite State Machine

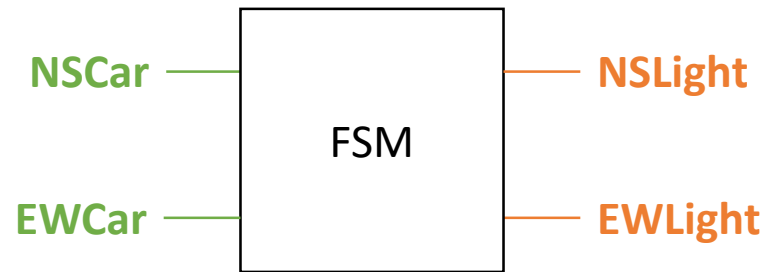


Finite State Machine



FSM Example

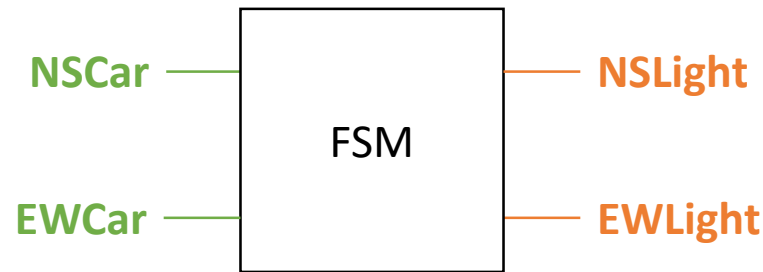
- Example: a finite state machine that controls a traffic signal at an intersection



- Inputs:
 - **NSCar**: there is a car on the north-south road
 - **EWCar**: there is a car on the east-west road
- Outputs:
 - **NSLight**: turn the north-south light green
 - **EWLight**: turn the east-west light green

FSM Example

- Example: a finite state machine that controls a traffic signal at an intersection



- Rules:
 - Rule #1: If a car is present on one road but not the other, turn green on the road where the car is
 - Rule #2: If a car is not present on either roads, stay green on the road that is currently green
 - Rule #3: If a car is present on both roads, turn green on the road that is not currently green

State Diagram

determine the number of states

Inputs:

NSCar

EWCar

Outputs:

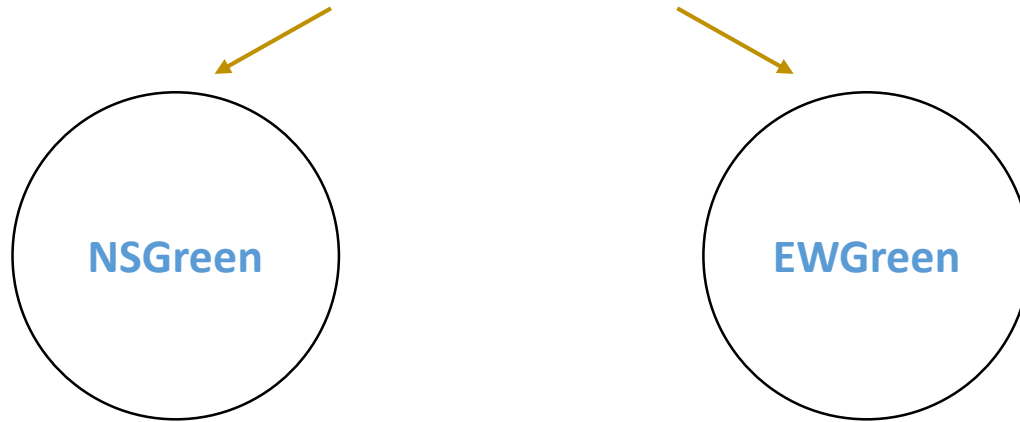
NSLight

EWLight

States:

NSGreen

EQGreen



State Diagram

Inputs:

NSCar

EWCar

Outputs:

NSLight

EWLight

States:

NSGreen

EWGreen



State Diagram

Inputs:

NSCar

EWCar

Outputs:

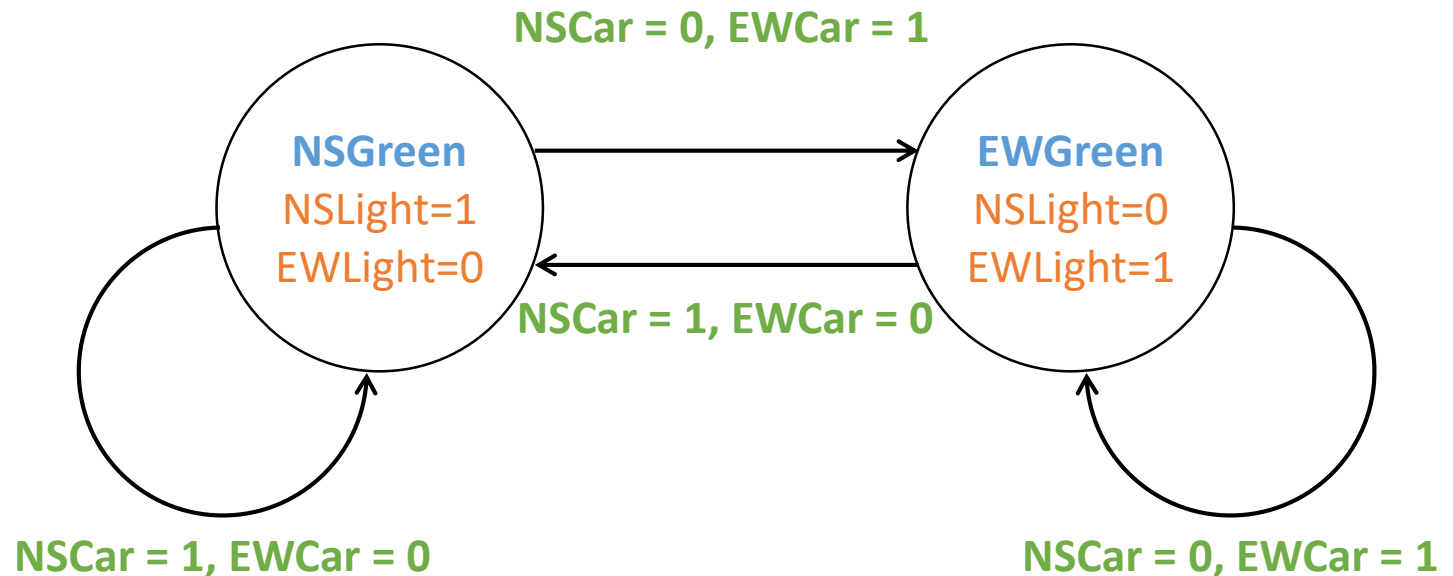
NSLight

EWLight

States:

NSGreen

EQGreen



determine transitions between states based on rules

Rule #1: If a car is present on one road but not the other, turn green on the road where the car is

State Diagram

Inputs:

NSCar

EWCar

Outputs:

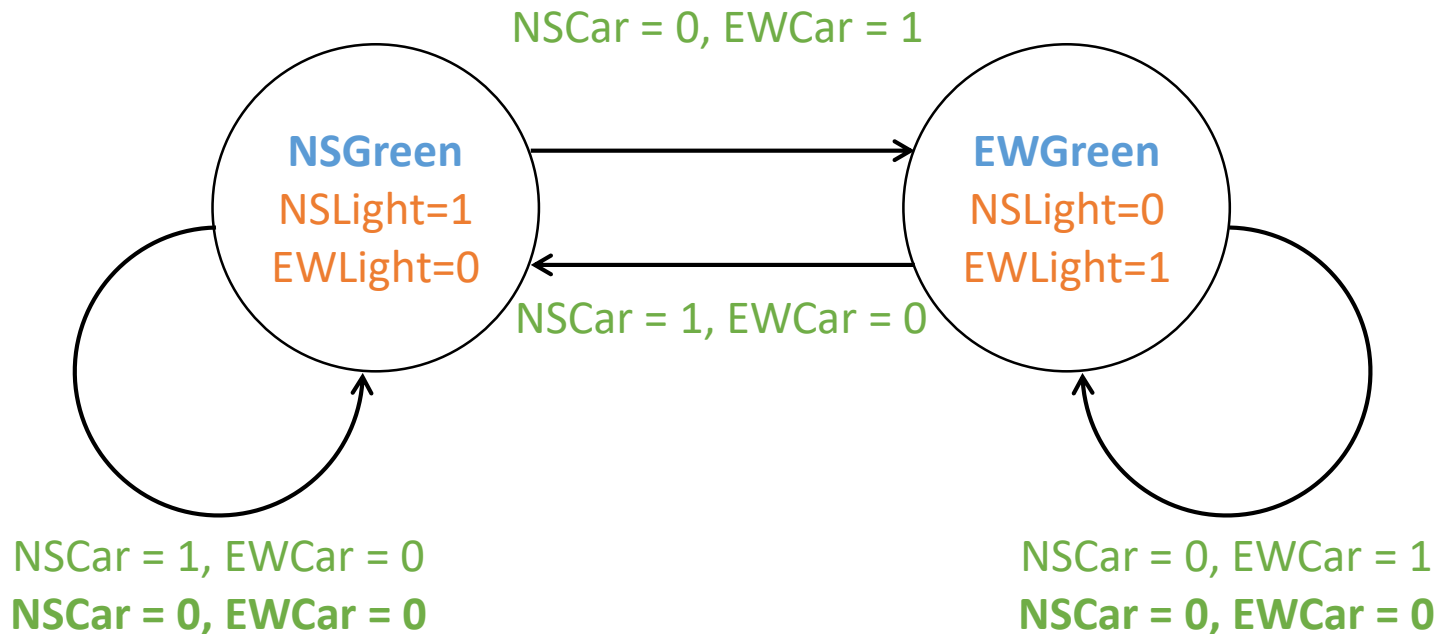
NSLight

EWLight

States:

NSGreen

EWGreen



determine transitions between states based on rules

Rule #2: If a car is not present on either roads,
stay green on the road that is currently green

State Diagram

Inputs:

NSCar

EWCar

Outputs:

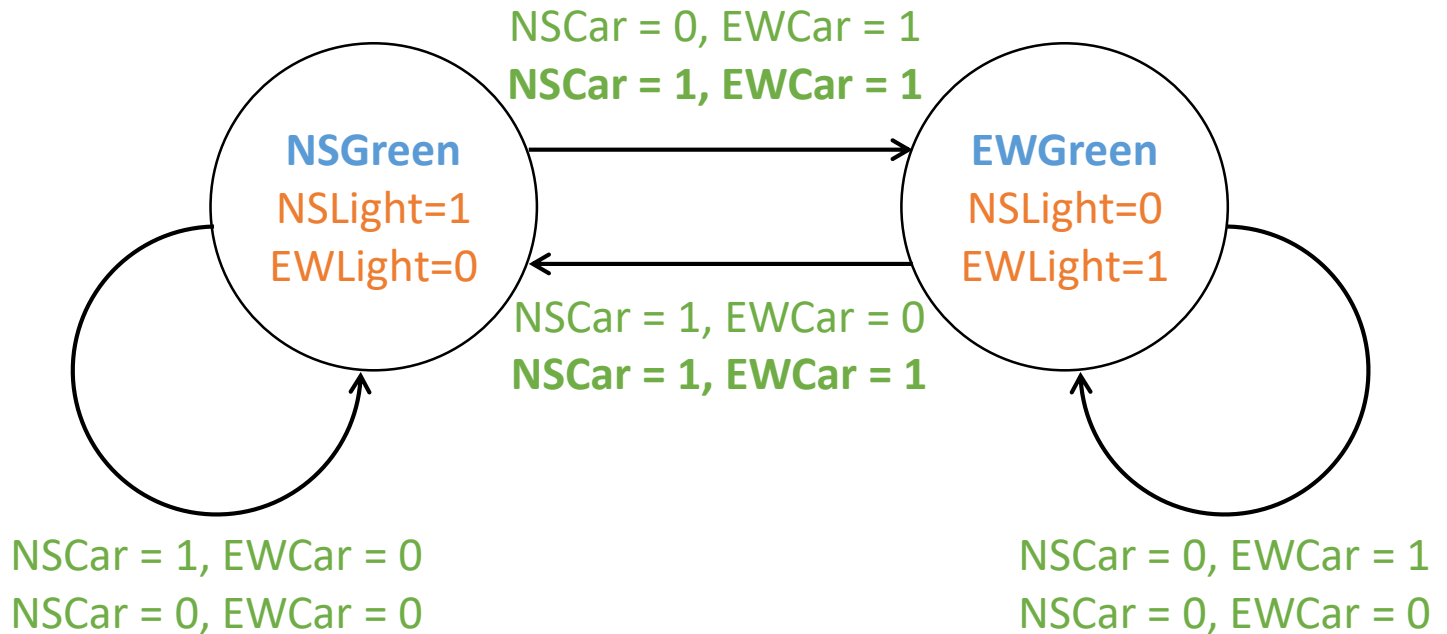
NSLight

EWLight

States:

NSGreen

EWGreen



determine transitions between states based on rules

Rule #3: If a car is present on both roads, turn green on the road that is not currently green

State Diagram

need 1 bit to represent 2 states
(in general $\lceil \log_2 N \rceil$ to represent N states)

Inputs:

NSCar

EWCar

Outputs:

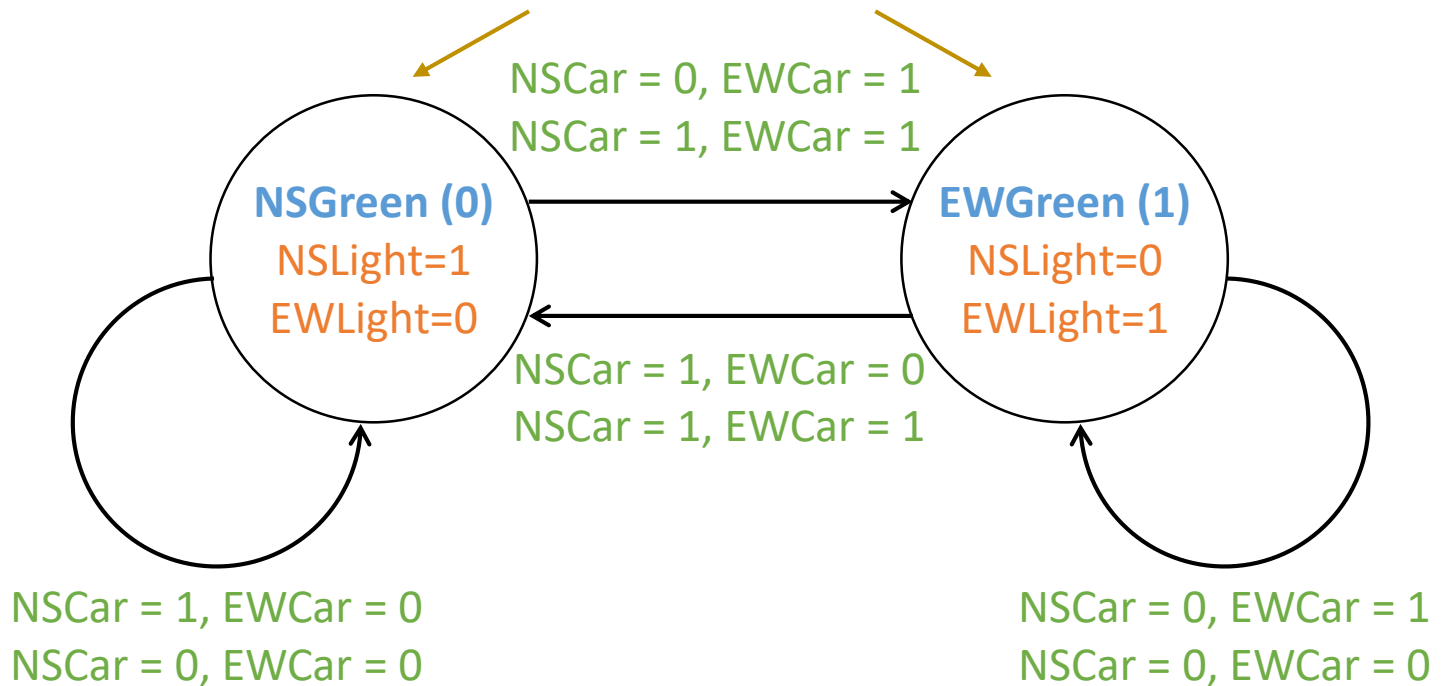
NSLight

EWLight

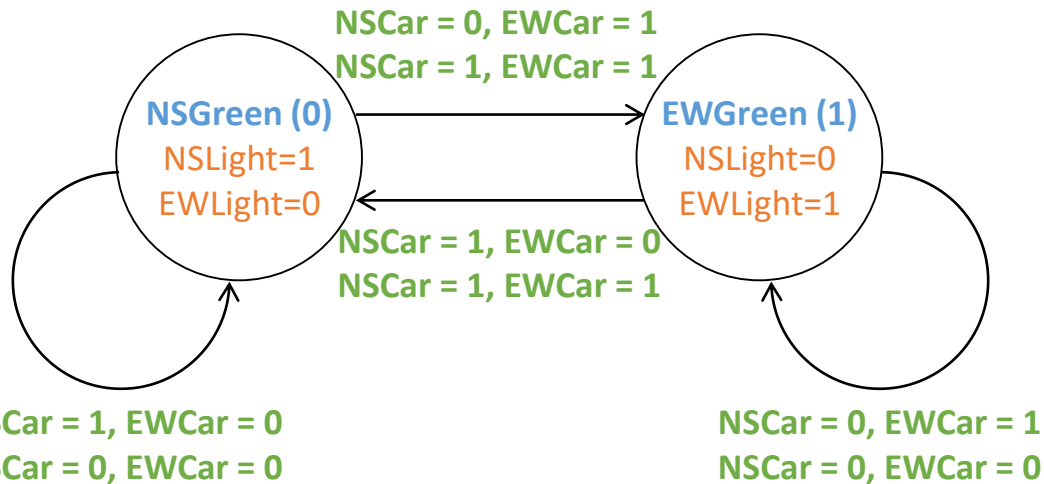
States:

NSGreen

EQGreen



Truth Table and Logic Equations



State _{current}	Outputs	
	NSLight	EWLight
0	1	0
1	0	1

State _{current}	Inputs		State _{next}
	NSCar	EWCcar	
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Output Function:

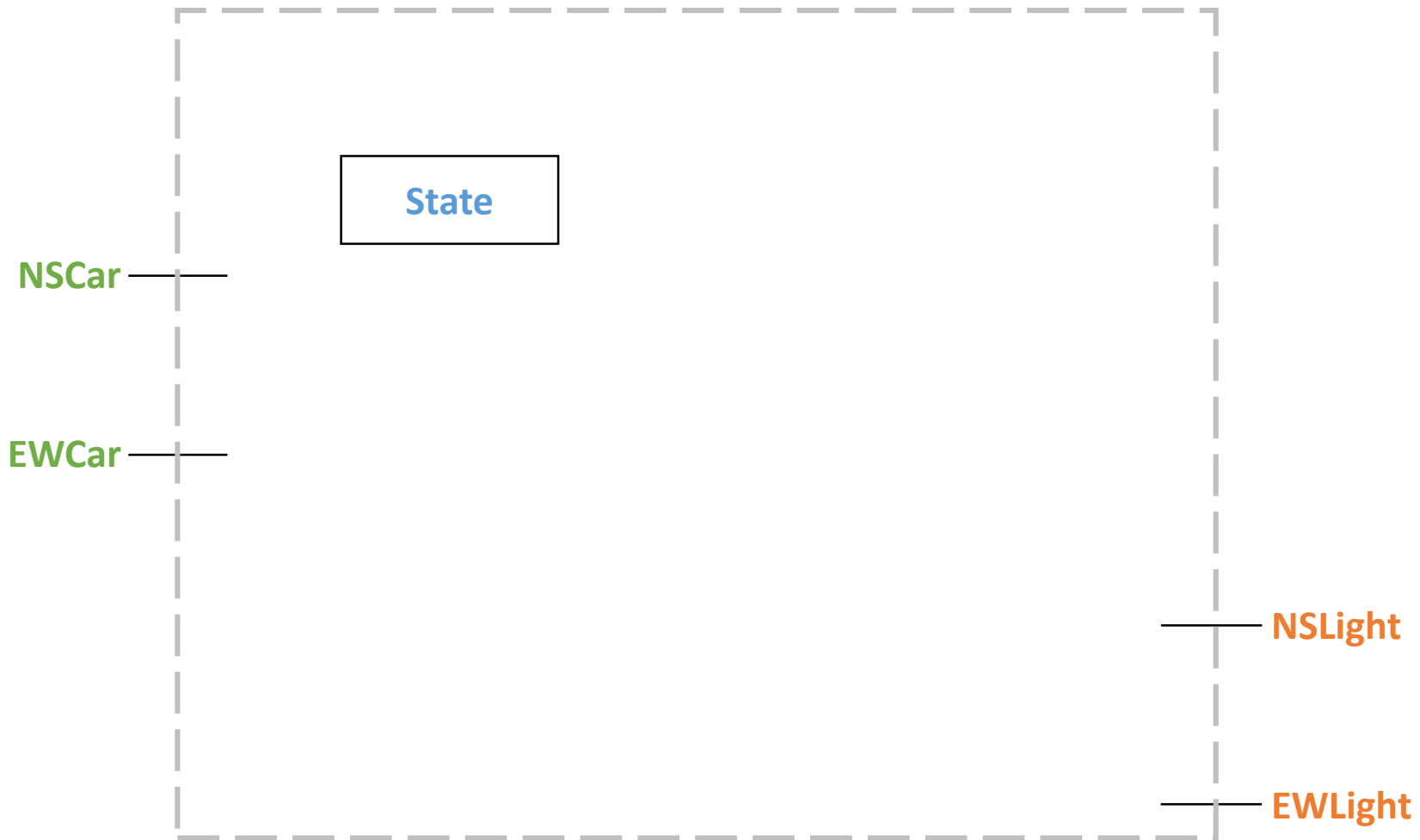
$$NSLight = \overline{State_{current}}$$

$$EWLight = State_{current}$$

Next-State Function:

$$State_{next} = State_{current} \cdot NSCar + \overline{State_{current}} \cdot EWCcar$$

Logic Diagram



Output Function:

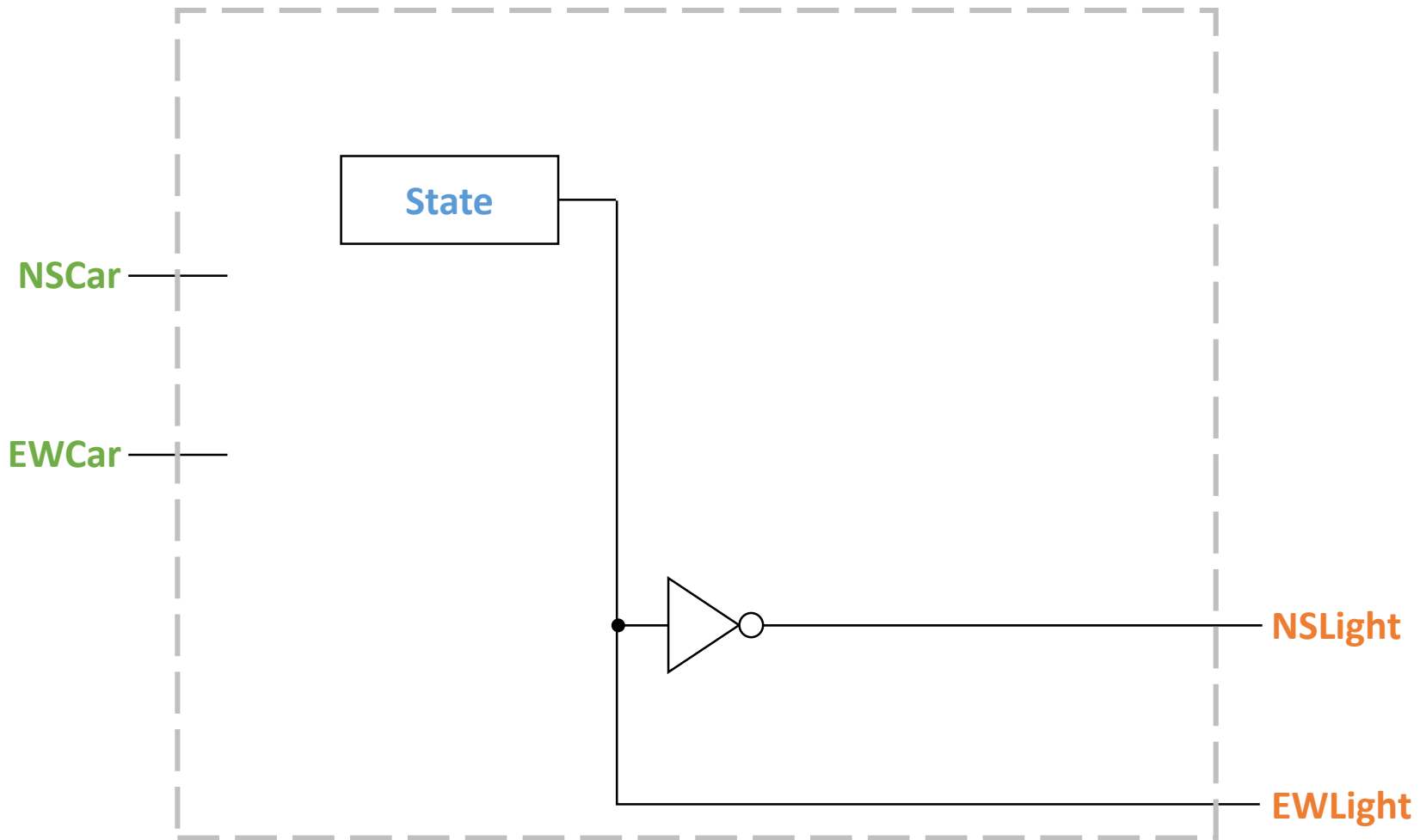
$$NSLight = \overline{State_{current}}$$

$$EWLight = State_{current}$$

Next-State Function:

$$State_{next} = State_{current} \cdot \overline{NSCar} + State_{current} \cdot EWCar$$

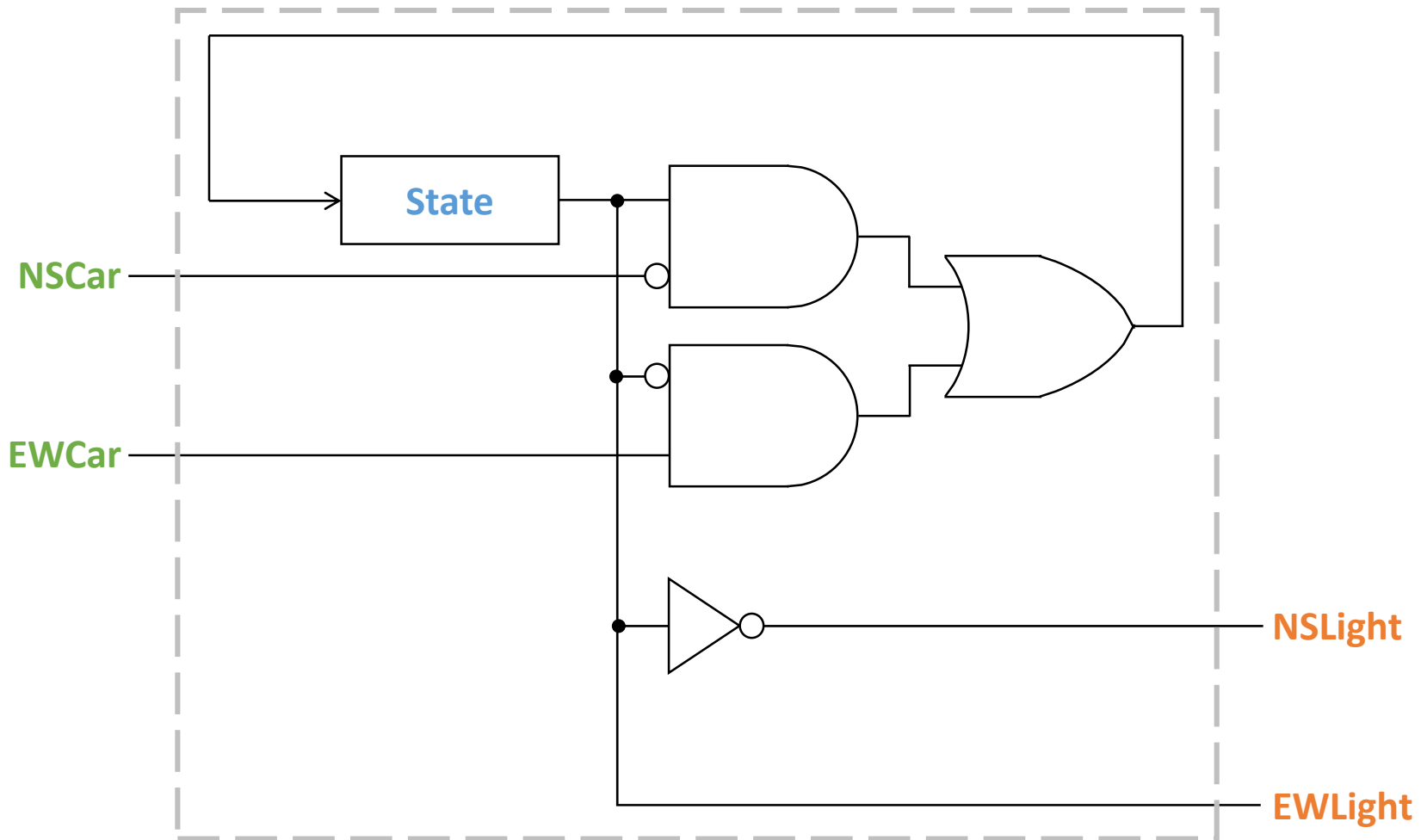
Logic Diagram



Output Function: $NSLight = \overline{State_{current}} \cdot NSCar$ $EWLigh = State_{current} \cdot EWCare$

Next-State Function: $State_{next} = State_{current} \cdot NSCar + \overline{State_{current}} \cdot EWCare$

Logic Diagram



Output Function:

$$NSLight = \overline{State_{current}}$$

$$EWLight = State_{current}$$

Next-State Function:

$$State_{next} = State_{current} \cdot \overline{NSCar} + State_{current} \cdot EWCar$$

Textbook Sections

- Some of the content in these slides corresponds to:
 - Textbook:
 - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
 - Sections:
 - B.10